RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

AD23532 – PRINCIPLES OF DATA SCIENCE
EXPERIMENT 3B: MODEL PLANNING AND BUILDING – REGRESSION

QUESTION:
A marketing agency wants to help a retail company optimize its advertising budget. The company invests in TV, Radio, and Newspaper advertising and wants to understand how these expenditures influence product sales. Build and compare Linear Regression, Polynomial Regression, and Support Vector Regression (SVR) models to predict sales, evaluate their performance using regression metrics, and provide recommendations for optimizing the advertising budget.

AIM:
To build and evaluate Linear Regression, Polynomial Regression, and Support Vector Regression models using the Advertising dataset, analyze relationships between advertising expenditure and sales, compare model performance using MAE, MSE, RMSE, and R^2 score, visualize actual versus predicted sales, and recommend suitable advertising budget strategies.

ALGORITHM:
1. Import the required Python libraries: Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.
2. Load the Advertising dataset into a Pandas DataFrame and display the first and last records, dataset shape, data types, missing values, duplicate records, and summary statistics.
3. Select TV, Radio, and Newspaper as input features (X) and Sales as the target variable (y).
4. Generate pair plots and a correlation heatmap to identify relationships between advertising expenditure and sales.
5. Split the dataset into 80% training data and 20% testing data.
6. Train a Linear Regression model and predict sales on the test dataset.
7. Train a Polynomial Regression model of degree 2 and predict sales on the test dataset.
8. Train a Support Vector Regression model using the RBF kernel and predict sales on the test dataset.
9. Evaluate all three models using MAE, MSE, RMSE, and R^2 score.
10. Compare model performance using a grouped bar chart and plot actual versus predicted sales for the best-performing model.
11. Identify the model with the lowest prediction error and suggest three recommendations for optimizing the advertising budget.

PROGRAM & OUTPUT:

```
# Name: Shyam M (241801267)
# Section 1: Import Required Libraries

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import (
    mean_absolute_error, mean_squared_error, r2_score
)
```

```
# Name: Shyam M (241801267)
# Section 2: Load and Explore Dataset

df = pd.read_csv("/content/advertising.csv")

print("First records:\n", df.head())
print("Last records:\n", df.tail())
print("Dataset shape:", df.shape)
print("\nData types:\n", df.dtypes)
print("\nMissing values:\n", df.isnull().sum())
print("Duplicate records:", df.duplicated().sum())
print("\nSummary statistics:\n", df.describe())

df = df.drop_duplicates()
df = df.dropna()
```

```
First records:
      TV  Radio  Newspaper  Sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3   12.0
3  151.5   41.3       58.5   16.5
4  180.8   10.8       58.4   17.9
Last records:
        TV  Radio  Newspaper  Sales
195   38.2    3.7       13.8    7.6
196   94.2    4.9        8.1   14.0
197  177.0    9.3        6.4   14.8
198  283.6   42.0       66.2   25.5
199  232.1    8.6        8.7   18.4
Dataset shape: (200, 4)

Data types:
 TV           float64
Radio        float64
Newspaper    float64
Sales        float64
dtype: object

Missing values:
 TV           0
Radio        0
Newspaper    0
Sales        0
dtype: int64
Duplicate records: 0

Summary statistics:
                TV       Radio   Newspaper       Sales
count  200.000000  200.000000  200.000000  200.000000
mean   147.042500   23.264000   30.554000   15.130500
std     85.854236   14.846809   21.778621    5.283892
min      0.700000    0.000000    0.300000    1.600000
25%     74.375000    9.975000   12.750000   11.000000
50%    149.750000   22.900000   25.750000   16.000000
75%    218.825000   36.525000   45.100000   19.050000
max    296.400000   49.600000  114.000000   27.000000
```

```
# Name: Shyam M (241801267)
# Section 3: Select Features and Target

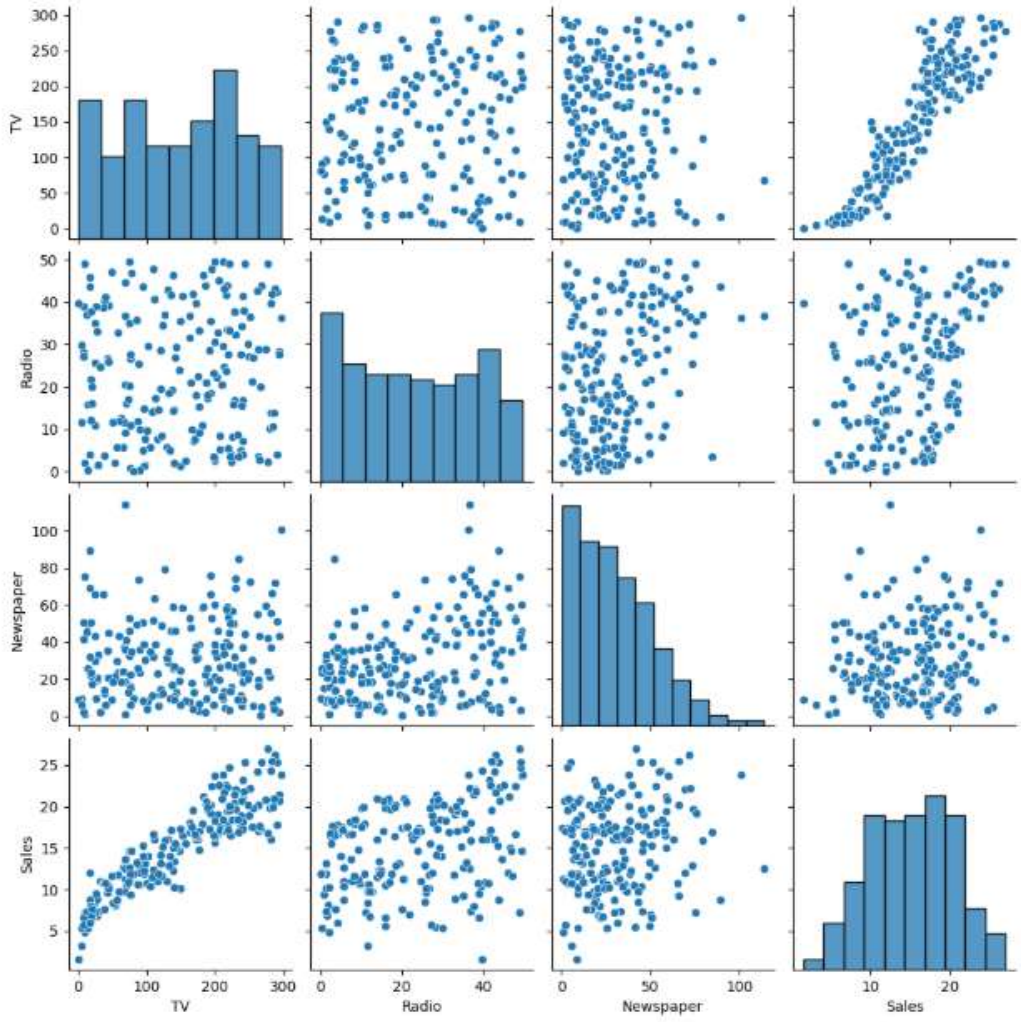
X = df[["TV", "Radio", "Newspaper"]]
y = df["Sales"]
```

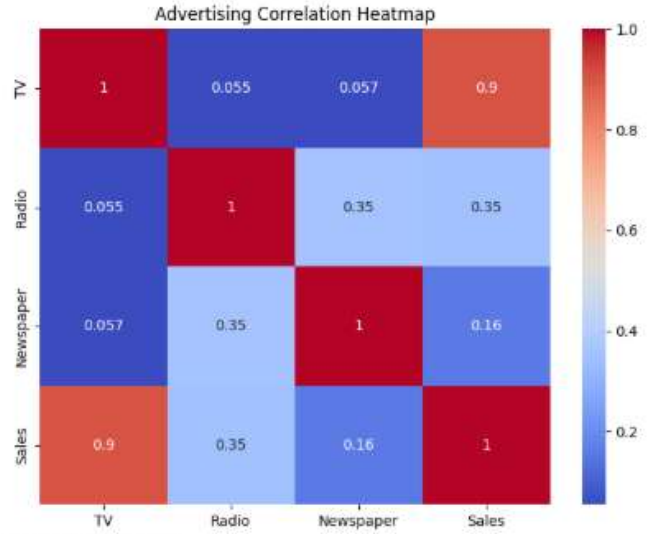
```
# Name: Shyam M (241801267)
# Section 4: Exploratory Data Analysis

sns.pairplot(df[["TV", "Radio", "Newspaper", "Sales"]])
plt.show()

plt.figure(figsize=(8, 6))
sns.heatmap(df[["TV", "Radio", "Newspaper", "Sales"]].corr(),
            annot=True, cmap="coolwarm")
plt.title("Advertising Correlation Heatmap")
plt.show()

print("Correlation with Sales:\n", df.corr(numeric_only=True)["Sales"])
```






```
Correlation with Sales:
 TV           0.901208
Radio         0.349631
Newspaper     0.157960
Sales         1.000000
Name: Sales, dtype: float64
```

```
# Name: Shyam M (241801267)
# Section 5: Split Training and Testing Data

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42
)
```

```python
# Name: Shyam M (241801267)
# Section 6: Build Regression Models

models = {
    "Linear Regression": LinearRegression(),

    "Polynomial Regression": make_pipeline(
        PolynomialFeatures(degree=2, include_bias=False),
        LinearRegression()
    ),

    "Support Vector Regression": make_pipeline(
        StandardScaler(),
        SVR(kernel="rbf")
    )
}

results = {}
predictions = {}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    predictions[name] = y_pred

    results[name] = {
        "MAE": mean_absolute_error(y_test, y_pred),
        "MSE": mean_squared_error(y_test, y_pred),
        "RMSE": np.sqrt(mean_squared_error(y_test, y_pred)),
        "R2 Score": r2_score(y_test, y_pred)
    }

    print("\n", name)
    print("MAE:", results[name]["MAE"])
    print("MSE:", results[name]["MSE"])
    print("RMSE:", results[name]["RMSE"])
    print("R2 Score:", results[name]["R2 Score"])
```

```
 Linear Regression
MAE: 1.2748262109549338
MSE: 2.9077569102710896
RMSE: 1.7052146229349223
R2 Score: 0.9059011844150826

 Polynomial Regression
MAE: 0.9033713382791676
MSE: 1.4425426369010854
RMSE: 1.2010589647894416
R2 Score: 0.9533174341074886

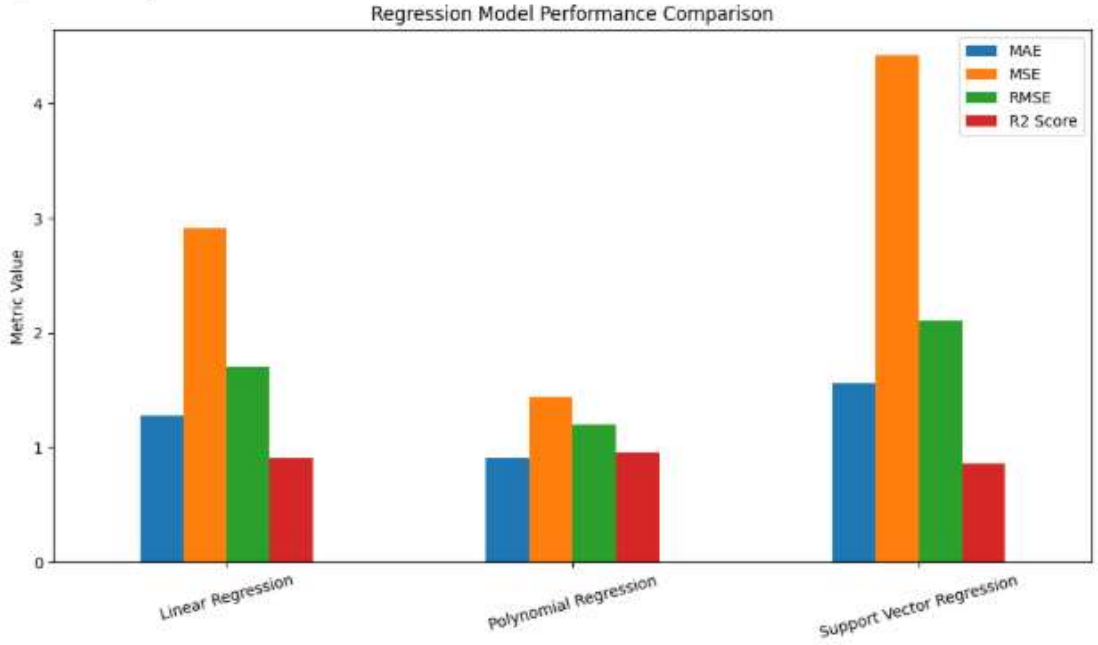
 Support Vector Regression
MAE: 1.5553546442382153
MSE: 4.421034260847772
RMSE: 2.102625563634137
R2 Score: 0.856929550700532 6
```

```
# Name: Shyam M (241801267)
# Section 7: Compare Model Performance

scores = pd.DataFrame(results).T
print("\nModel Comparison:\n", scores)

scores.plot(kind="bar", figsize=(10, 6))
plt.title("Regression Model Performance Comparison")
plt.ylabel("Metric Value")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

```
Model Comparison:
                             MAE       MSE      RMSE  R2 Score
Linear Regression       1.274826  2.907757  1.705215  0.905901
Polynomial Regression   0.903371  1.442543  1.201059  0.953317
Support Vector Regression  1.555355  4.421034  2.102626  0.856930
```




```
# Name: Shyam M (241801267)
# Section 8: Identify Best Model

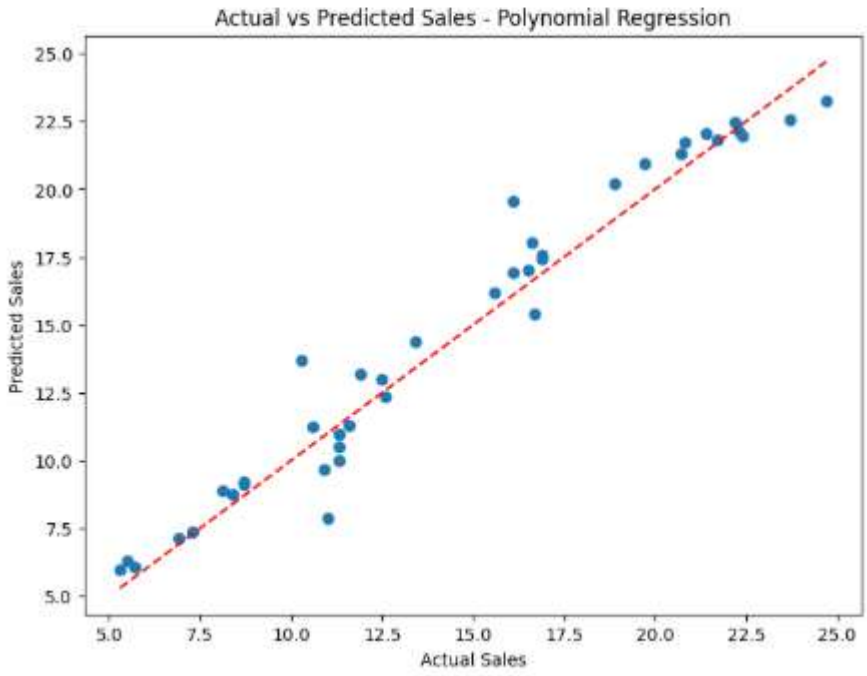
best_model = min(results, key=lambda m: results[m]["RMSE"])
best_pred = predictions[best_model]

print("Best Model:", best_model)
print("Lowest RMSE:", results[best_model]["RMSE"])
```

```
Best Model: Polynomial Regression
Lowest RMSE: 1.2010589647894416
```

```
# Name: Shyam M (241801267)
# Section 9: Actual vs Predicted Sales

plt.figure(figsize=(8, 6))
plt.scatter(y_test, best_pred)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    "r--"
)
plt.xlabel("Actual Sales")
plt.ylabel("Predicted Sales")
plt.title("Actual vs Predicted Sales - " + best_model)
plt.show()
```




```
# Name: Shyam M (241801267)
# Section 10: Business Recommendations

print("\nBusiness Recommendations:")
print("1. Allocate more budget to advertising channels with stronger sales relationships.")
print("2. Review channel performance regularly before changing advertising spending.")
print("3. Use the best-performing model to support sales forecasting and budget planning.")
```

```
Business Recommendations:
1. Allocate more budget to advertising channels with stronger sales relationships.
2. Review channel performance regularly before changing advertising spending.
3. Use the best-performing model to support sales forecasting and budget planning.
```

RESULT:

Thus, Linear Regression, Polynomial Regression, and Support Vector Regression models were successfully built and evaluated using the Advertising dataset. Their performance was compared using MAE, MSE, RMSE, and R^2 score. The best-performing model was identified based on the lowest RMSE, and actual versus predicted sales were visualized. Recommendations were provided to help optimize advertising expenditure and improve sales forecasting.